BrainAI

ProSuite Industrial

PROSUITE INDUSTRIAL

PRODUKT-DOKUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	**PROSUITE INDUSTRIAL**	**🟢 ÖFFENTLICHE KLASSIFIZIERUNG**
Core Architecture & Development	**Technical Documentation**	Zugriffsstufe: Öffentlich / Beta

---

Revisionshistorie

Version	Datum	Autor	Status	Beschreibung
1.0.0	28.02.2026	S. Köhne	Stable	Initial Release: ProWB Markdown Parser (GFM) & Security Spec

1. Einleitung: ProWB Markdown Engine

Der **ProWB Markdown Parser** (`md_parser.c`) ist die proprietäre Text-Rendering-Engine des Pro Web Builders. Seine primäre Aufgabe ist die verlustfreie und hochsichere Konvertierung von Dokumentationen (Handbüchern, Audit-Reports) in W3C-konformes HTML5.

Da der ProWB in Hochsicherheitsumgebungen (ProSuite Industrial) eingesetzt wird, wurde der Parser von Grund auf in striktem C geschrieben. Er verzichtet auf unsichere Regex-Bibliotheken und arbeitet stattdessen mit deterministischer Pointer-Arithmetik, um maximale Speicher- und Ausführungssicherheit zu garantieren. Er unterstützt den **GitHub Flavored Markdown (GFM)** Industriestandard.

2. Sicherheitsarchitektur (Military Grade)

Ein Parser, der externe oder dynamisch generierte Dateien verarbeitet, ist ein primäres Ziel für Puffer-Angriffe und Code Injections. Die Engine ist auf C-Ebene gegen folgende Angriffsvektoren gehärtet:

2.1 Speichersicherheit (Memory Bounds)

- **Buffer Overrun Protection:** Strings und Arrays haben harte, nicht überschreitbare Grenzen. Die Zeilenlänge ist strikt auf `LINE_SIZE` (8.192 Bytes) limitiert. Tabellen sind auf maximal 32 Spalten (`MAX_TABLE_COLS`) begrenzt. Versucht ein Angreifer größere Payloads zu injizieren, schneidet C den Puffer physisch ab.
- **Buffer Underrun Protection:** Vor jedem Pointer-Zugriff (z.B. `clean_name[2]`) wird die absolute Mindestlänge des Strings via `strlen()` validiert. Negative oder zu kurze Längen führen zum sofortigen Abbruch der Funktion.
- **OOM (Out-of-Memory) Handling:** Jede dynamische Speicherzuweisung (`malloc`) wird auf `NULL` geprüft. Bei Speichererschöpfung bricht das Rendering des betroffenen Elements ab, ohne einen System-Segfault zu provozieren.

2.2 XSS & Code Injection Prevention

- **ID-Sanitization (`_sanitize_for_id`):** Dateinamen und IDs für JavaScript-Events (`onclick`) werden durch einen aggressiven Filter gezwungen. Nur alphanumerische Zeichen (`A-Z`, `a-z`, `0-9`) und Unterstriche (_) sind erlaubt. Manipulierte IDs wie `view_home')alert(1` werden deterministisch in `view_home___alert_1` umgewandelt.
- **HTML-Escaping (`_escape_html_string`):** Potenziell gefährliche Steuerzeichen (`<`, `>`, `&`, `"`, `'`) in Code-Blöcken oder Labeln werden konsequent in ihre sicheren HTML-Entitäten (`<`, `>` etc.) maskiert.

3. Unterstützte GFM-Syntax (Reference)

Die Engine unterstützt die folgenden GitHub Flavored Markdown (GFM) Auszeichnungen.

3.1 Überschriften (Headings)

Überschriften werden von H1 bis H6 unterstützt. **Wichtig:** Nach dem `#` muss zwingend ein Leerzeichen folgen, andernfalls wird es als normaler Text (Hashtag) gewertet.

```
# Level 1 (H1)
## Level 2 (H2)
###### Level 6 (H6)
```

3.2 Textformatierung

Unterstützt fette, kursive und durchgestrichene Formatierungen, auch in Kombination.

```
**Fetter Text** oder __Fetter Text__
*Kursiver Text* oder _Kursiver Text_
~~Durchgestrichener Text~~ (Strikethrough)
```

3.3 Listen & Aufzählungen

Die Engine schaltet intelligent zwischen geordneten (`<ol>`) und ungeordneten (`<ul>`) Listen um. Ebenfalls werden Task-Listen nativ in deaktivierte HTML-Checkboxen konvertiert.

```
- Ungeordnete Liste (Strich)
* Ungeordnete Liste (Stern)

1. Geordnete Liste (Punkt)
```

```
1) Geordnete Liste (Klammer)

- [ ] Offene Aufgabe (Task)
- [x] Abgeschlossene Aufgabe (Task)
```

3.4 Blockquotes (Zitate)

```
> Dies ist ein hervorgehobener Zitat-Block.
> Er kann sich über mehrere Zeilen erstrecken.
```

3.5 Code-Blöcke & Inline-Code

Inline-Code wird durch einfache Backticks markiert. Mehrzeiliger Code unterstützt Language-Tags (z. B. `c`, `javascript`) für späteres Syntax-Highlighting.

```
Nutzen Sie die `_malloc` Funktion für Speicher.

```c
int main() {
    printf("ProSuite Industrial");
    return 0;
}
```
```

(Hinweis: Den obigen Code-Block im Markdown natürlich mit echten Backticks schließen)

3.6 Links & Bilder

```
[Besuchen Sie BrainAI](https://brainai.example)
![ProSuite Logo](./assets/logo.png)
```

3.7 Tabellen (mit Alignment)

GFM-Tabellen werden vollständig geparst. Die Doppelpunkte in der Trennlinie definieren die Ausrichtung (Links, Zentriert, Rechts).

```
| Modul | Status | Metrik |
| :--- | :---: | ---: |
| ProTU | Aktiv | 48-52% |
| ProEDC | Aktiv | 256-Bit |
```

3.8 Trennlinien

Erzeugt eine horizontale Linie (`<hr>`).

```
---
```

© 2026 Sascha Köhne	✉️ **Contact for KYC: koehne83@googlemail.com**	🛡️ **ProTU-Verified**
System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**